

# Text Editor Platform Cursor Add, Delete, Move — Production System

Design Date: 2025-08-10 UTC

Text Editor Platform Cursor Add, Delete, Move — Production System

Design Date: 2025-08-10 UTC

One■Pager Summary Goal Deliver a cross■platform text editor with fast local edits (insert, delete, move cursor), durable persistence, optional cloud sync, and enterprise■grade reliability. The platform must support single■user local editing first, with pathways to collaborative editing and extensions without re■architecting.

## Key Outcomes

Sub■ten millisecond local edit and cursor move latencies on commodity hardware.  
Crash■safe persistence with zero data loss for acknowledged saves and low overhead autosave.  
Extensible core that can swap editor buffers (gap buffer, piece table, rope) based on file size and workload. Secure multi■tenant cloud services for sync, settings, and backups with clear SLOs and audits.

## Primary Assumptions

Files from bytes to multi■gigabyte logs; typical dev and prose workloads under ten megabytes. Single■user edits in MVP; future optional collaboration via OT or CRDT. Targets: desktop native and web. Plug■in API for language features, theming, and keymaps.

## Functional Requirements

Editing Insert, delete, replace, undo and redo, cursor and selection move, multi■cursor, copy and paste, find and replace. Files Open large files in streaming mode; encoding detection; line endings; autosave and recovery. Navigation Go to line, symbol, bracket matching, scroll, markers and bookmarks. State Undo or redo stack, change history, dirty flags, per■file settings and metadata. Sync and Backup optional Cloud backup and settings sync; conflict detection and merge prompts. Extensibility Plug■in API for language servers, formatters, linters, themes, keybindings, and renderers. Accessibility Screen reader hints, high contrast themes, keyboard■first UX.

Non■

Local p95 latency for insert, delete, and move under ten milliseconds for typical files; under fifty milliseconds for megabyte■scale edits. Startup to first paint under five hundred milliseconds for recent files. Durability RPO zero for explicit save; autosave every two seconds with journaling. Availability for cloud endpoints ninety nine point nine five percent; metadata writes ninety nine point nine percent. Privacy and security aligned to SOC two, GDPR, CCPA when cloud sync is enabled.

## Architecture Overview

Client Core Editor process owns the buffer, cursor model, viewport, rendering, and undo or redo. Rendering thread is decoupled from the buffer via immutable snapshots. Extension Host Isolated plugin sandbox with message passing to avoid UI stalls. Language features over LSP style RPC. Persistence Local document store uses journaled saves and crash recovery files. Cloud sync uses content■addressed chunks and delta uploads. Control Plane optional Tenant and user management, device list, key escrow for encrypted backups, policy and feature flags. Data Plane optional Sync workers, object storage for file chunks, metadata database for file trees and versions, event streams for devices. Editor Buffer Design Small to medium files Gap buffer for very fast local insert and delete near the cursor and low memory overhead. Large and fragmented edits Piece table to treat original and append■only buffers with a piece index for edits. Enables O(1) insert cost amortized and cheap undo or redo. Very large files and multi■pane editors Rope or B tree of leaf chunks for logarithmic inserts and deletes, efficient substring and split or join; supports background rebalancing. Selections and cursors Model single and multiple carets with anchors that survive edits via marker trees; use tombstones during complex operations. Undo or redo Command log with snapshots every N operations or on large edits; inverse

operations recorded with cursor restore.

## Storage Strategy

Local Journaled writes plus last known good file; compaction on idle. Recovery files kept per document with time based retention. Cloud optional Object storage for chunks with lifecycle policies; metadata in relational store for consistency; search and activity feed in analytical store. Indexing Per document line index and token cache for rendering; optional trigram or suffix index for large docs. Scaling Approach Client Use shared nothing design per file tab; background workers for indexing, tokenization, and saves. Zero copy snapshots for renderer to avoid stalls. Cloud Shard metadata by tenant id and document id; chunks partitioned by hash prefix; sync workers autoscale on queue depth and egress. Caching Local hot line cache, render tile cache, and glyph atlas cache. In cloud, CDN for chunk downloads and signed URLs.

## Data Models

(abridged) Document doc id, tenant id, path, encoding, language hint, creation and update timestamps, size, version head. Chunk optional for sync hash, size, compression flag, storage pointer, refcount, created at. Version doc id, version id, parent id, author device id, summary, timestamp, conflict flags. Edit op doc id, op id, sequence, pos, length, payload size, type insert or delete or replace, cursor before and after, created at. Settings user id, key, value, device scope, updated at. Audit actor id, action, target id, timestamp, diff summary.

## Latency Budgets and SLAs

Local Keypress to render under ten milliseconds p95 for typical files; incremental reflow under eight milliseconds per frame at sixty frames per second. Save to disk under fifty milliseconds for small files and proportional for large files via streaming. Cloud Upload commit under five hundred milliseconds for small files; large files through chunked parallel uploads. List or metadata reads under one hundred milliseconds p95. SLA Cloud read ninety nine point nine five percent, write ninety nine point nine percent; SLOs for sync freshness under five seconds p95.

Security, Reliability, Compliance Local security Sandbox plugins, limit file system scope, safe clipboard handling, and signed updates. Cloud security optional TLS one point three, KMS encryption at rest and per tenant keys, OAuth or OpenID Connect for auth, RBAC and ABAC for access control, signed URLs for chunks. Privacy and compliance Data minimization, regional residency options, consent and DSAR APIs, immutable audit logs, and retention controls. Reliability Crash resistant local saves, cloud multi AZ and versioned objects, replayable event logs, disaster recovery with tested restores.

Multi-Tenancy Tenant id carried in every cloud key; quotas for storage, bandwidth, and API rate. Optional hard isolation per enterprise customer with dedicated buckets and VPCs.

CI or CD and Infrastructure as Code Everything as code for cloud components using Terraform or Pulumi; Helm or Argo CD for Kubernetes; SAST or DAST; image signing and provenance; canary rollouts and automatic rollback on SLO burn.

APIs and Integration Points Local Plug-in host APIs for buffer edits, cursor moves, selections, markers, and folding; render and keymap APIs; LSP bridge. Cloud optional REST APIs for auth, list files, get and put chunks, commit version, get versions, diff, restore, search, and audit export. Desktop or Web Auto updater, crash reporter, and telemetry with opt in.

## Key Algorithms and Workflows

Insert text Translate screen selection to buffer coordinates, edit through buffer structure, update marker tree, coalesce edits for undo, enqueue repaint of dirty lines only. Delete

text Range delete maps to piece removal or gap compaction; maintain grapheme cluster boundaries for Unicode correctness. Move cursor Logical movement respects UTF-8 or UTF-16 code units and grapheme clusters; physical movement uses layout cache. Word and line navigation use lexical rules. Search Two-tier: fast line index for simple substring and incremental suffix or trigram index for large files; highlight via incremental reflow. Autosave and recovery Every two seconds flush delta to a journal file; on crash, recover by replaying journal onto last stable snapshot. Cloud sync optional Chunk file into

content■addressed blocks, compute rolling checksums, upload only changed chunks, maintain version DAG and conflict markers; present merge UI if concurrent edits detected.

## Observability and Operations

Local Frame time, input to paint latency, undo or redo size, autosave duration, crash reports with minimal data. Cloud Golden dashboards for p95 latency per endpoint, error budgets, queue depth, object storage errors, sync backlog, and cost guardrails. OpenTelemetry traces and structured logs with tenant id. Runbooks Corrupted journal recovery, chunk reupload, metadata rollback, and canary disable.

## Capacity Planning and Cost Estimates

Local Memory target under two hundred megabytes per medium document with rope or piece table; GPU budget for rendering under five percent on integrated graphics. Cloud Illustrative month for one hundred thousand active users, average one gigabyte storage each, twenty five percent monthly churn, and sync traffic one to three terabytes per day. Monthly cost range depends on storage class, egress, and CDN, on the order of tens of thousands.

## Testing Strategy and Failure Drills

Editor correctness Unit tests for cursor movement and grapheme boundaries, insertion and deletion, undo or redo, and large file streaming. Fuzzers for parser edge cases and encodings. Performance Microbenchmarks for buffer ops on representative files; scroll and render frame tests; soak tests for memory fragmentation. Resilience Power cut tests during save and autosave; crash and recovery; cloud chaos drills for object store, queue, and database failover. Security Sandbox escape tests, plugin permission checks, dependency scans, and signed update verification.

## Phased Build Plan

MVP three months Single■user editor with gap buffer, undo or redo, autosave journal and recovery, find or replace, theming, plugin host skeleton, and minimal telemetry. Phase two six to nine months Piece table for large docs, ropes for huge files, extension sandbox, LSP, richer rendering pipeline and line wrapping, cloud backup and settings sync, signed updates. Phase three twelve months Collaboration mode via OT or CRDT, offline merges with conflict UI, enterprise tenancy with per tenant keys, SSO, policy controls, and advanced observability.

Key Choices, Trade■Offs, Risks, Mitigations Gap buffer is blazing fast near the cursor but degrades for dispersed edits; piece table and rope address that at higher complexity. Immutable snapshots simplify rendering but increase memory; mitigate with chunked snapshots and pooling. Content■addressed chunks cut bandwidth but require garbage collection and refcounts. Plugins add power but risk instability; mitigate with isolated sandboxes and watchdogs.